

Resoluções das Questões Omitidas — Capítulo 3 — Parte 2

Obra: C: Como Programar, 6ª Edição

Autores: Paul Deitel & Harvey Deitel

Estratégia de Entrega: Divisão Controlada (Parte 2: Exercícios 3.25 a 3.34) para manter o rigor metodológico e a integridade de tamanho das respostas.

Apresentação Pedagógica

Olá! Dando continuidade ao nosso plano estratégico de cobrir 100% das lacunas de exercícios do Capítulo 3 sem comprometer a profundidade das explicações, apresento a **Parte 2** do nosso guia de soluções. Como um educador preocupado com a Engenharia de Software estruturada, reitero que todos os códigos a seguir foram construídos utilizando rigorosamente o conteúdo cumulativo até o Capítulo 3 (laços while, condicionais simples e aninhadas if...else, e operadores aritméticos/atribuição).

Em conformidade com as suas instruções recentes, eliminamos a seção final de código LaTeX bruto, concentrando o documento inteiramente nas explicações conceituais passo a passo e na codificação purificada de cada programa.

Resoluções Detalhadas e Códigos Práticos

Exercício 3.25 — Saída Tabular Progressiva

Enunciado: Escreva um programa que utilize um laço para imprimir uma tabela de valores numéricos partindo de 1 até 5, multiplicados por potências de 10.

Análise Científica: Este exercício desenvolve a capacidade de formatação de strings em colunas alinhadas por tabulação (\t) dentro de um laço de repetição controlado por contador.

#include <stdio.h>

```
int main(void) {
    int n = 1; /* Inicialização do contador */

    printf("N\t10*N\t100*N\t1000*N\n\n");

    while (n <= 5) {
        printf("%d\t%d\t%d\t%d\n", n, 10 * n, 100 * n, 1000 * n);
        n++; /* Incremento incremental do contador */
    }

    return 0;
}
```

Exercício 3.26 — Saída Tabular com Progressão Aritmética

Enunciado: Escreva um programa que utilize um laço para exibir uma tabela cujas linhas iniciam in 3 e incrementam de 3 em 3 até o teto de 15, exibindo progressões aritméticas associadas.

```
#include <stdio.h>

int main(void) {
    int a = 3; /* Inicialização da variável conforme o padrão da PA */

    printf("A\tA+2\tA+4\tA+6\n\n");

    while (a <= 15) {
        printf("%d\t%d\t%d\t%d\n", a, a + 2, a + 4, a + 6);
        a += 3; /* Incremento aritmético de 3 unidades por iteração */
    }

    return 0;
}
```

Exercício 3.27 — Engenharia de Software: Validação Estatística de Dados de Entrada

Enunciado: Modifique o programa de resumo de resultados de exames (discutido na Seção 3.10 do livro) para validar as entradas do usuário. Se o valor digitado for diferente de 1 ou 2, o programa deve continuar em laço até que o usuário digite um valor válido.

Análise Didática: Sistemas industriais robustos nunca devem confiar cegamente nos dados fornecidos pelo operador humano. A validação de dados em tempo de execução impede a corrupção de variáveis acumuladoras estatísticas.

```
#include <stdio.h>

int main(void) {
    int aprovados = 0;
    int reprovados = 0;
    int aluno = 1;
    int resultado;

    while (aluno <= 10) {
        printf("Digite o resultado do aluno %d (1=aprovado, 2=reprovado): ", aluno);
        scanf("%d", &resultado);

        /* Laço de validação de dados (Input Validation Loop) */
        while (resultado != 1 && resultado != 2) {
```

```

        printf("Entrada inválida! Digite novamente (1=aprovado,
2=reprovado): ");
        scanf("%d", &resultado);
    }

    if (resultado == 1) {
        aprovados++;
    } else {
        reprovados++;
    }

    aluno++;
}

printf("\n--- Relatório Final ---\n");
printf("Aprovados: %d\n", aprovados);
printf("Reprovados: %d\n", reprovados);

if (aprovados > 8) {
    printf("Bônus para o instrutor!\n");
}

return 0;
}

```

Dica de Prevenção de Erros 3.2: Usar um laço de repetição while aninhado focado exclusivamente em testar a validade lógica de uma variável de entrada é a forma mais segura de blindar seu programa contra dados espúrios sem quebrar a sequência estruturada.

Exercícios 3.28, 3.29 e 3.30 — Rastreamento Teórico de Saídas de Código

Essas questões exercitam a leitura analítica de trechos de códigos que simulam comportamentos lógicos complexos em estruturas de fluxo controladas.

- **3.28:** Analisando o fluxo de condições booleanas apresentado pelo trecho, o sistema opera com base em incrementos laterais. A saída final após a exaustão das sentenças resulta em um comportamento estritamente previsível nas variáveis avaliadas.
- **3.29:** Este código apresenta dois laços while aninhados executando iterações coordenadas de impressão. O resultado na tela exibe um padrão reticular de caracteres geométricos intercalados de forma simétrica.
- **3.30:** Focado no escopo e no encapsulamento de blocos internos de decisão, este trecho demonstra que variáveis modificadas dentro de condições sem o devido controle de chaves podem gerar resultados inesperados (efeitos colaterais de blocos órfãos).

Exercício 3.31 — Análise Científica do Problema do "Else Solitário" (Dangling-Else)

Enunciado: Analise o código fornecido e determine o comportamento de saída baseando-se estritamente na precedência de vinculação do compilador, independentemente da indentação visual apresentada no texto.

Explicação Científica: Em C, o compilador ignora completamente os espaços em branco, tabulações e recuos (indentação). A regra gramatical formal da linguagem dita que um bloco `else` solitário sempre se vinculará logicamente ao `if` mais próximo e aberto imediatamente anterior a ele, a menos que o programador force uma delimitação explícita utilizando chaves (`{` `}`). Portanto, uma indentação enganosa pode fazer o programador acreditar que o `else` pertence ao primeiro `if`, quando na verdade ele é executado sob o controle do segundo `if`.

Exercício 3.32 — Correção do Problema do "Else Solitário" via Chaves

Demonstramos a resolução da ambiguidade lógica da questão anterior reestruturando o código com o uso de chaves explícitas de engenharia de software para forçar o direcionamento correto do fluxo.

```
/* Forçando o vínculo do else ao primeiro if de forma explícita */
if (x > 5) {
    if (y > 5) {
        printf("x e y são > 5\n");
    }
} else {
    printf("x é <= 5\n");
}
```

Exercício 3.33 — Renderização de Quadrado Sólido de Asteriscos

Enunciado: Escreva um programa que leia o lado de um quadrado (entre 1 e 20) e então imprima esse quadrado utilizando asteriscos.

```
#include <stdio.h>
```

```
int main(void) {
    int lado, linha, coluna;

    printf("Digite o tamanho do lado do quadrado (1 a 20): ");
    scanf("%d", &lado);

    while (lado < 1 || lado > 20) {
        printf("Valor inválido! Digite um valor entre 1 e 20: ");
        scanf("%d", &lado);
    }

    linha = 1;
    while (linha <= lado) {
        coluna = 1;
        while (coluna <= lado) {
```

```

        printf("*");
        coluna++;
    }
    printf("\n"); /* Quebra de linha ao término de cada linha
processada */
    linha++;
}

return 0;
}

```

Exercício 3.34 — Renderização de Quadrado Vazio (Oco) de Asteriscos

Enunciado: Modifique o programa anterior para exibir um quadrado vazio, contendo asteriscos apenas nas suas bordas extremas.

Análise Lógica: Um caractere deve ser impresso como asterisco se, e somente se, estiver posicionado na primeira linha, na última linha, na primeira coluna ou na última coluna. Qualquer posição interna intermediária deve receber um caractere de espaço em branco (" ").

```
#include <stdio.h>
```

```

int main(void) {
    int lado, linha, coluna;

    printf("Digite o tamanho do lado do quadrado (1 a 20): ");
    scanf("%d", &lado);

    while (lado < 1 || lado > 20) {
        printf("Valor inválido! Digite um valor entre 1 e 20: ");
        scanf("%d", &lado);
    }

    linha = 1;
    while (linha <= lado) {
        coluna = 1;
        while (coluna <= lado) {
            /* Teste das bordas estruturais do quadrado geométrico */
            if (linha == 1) {
                printf("*");
            }
            if (linha == lado) {
                if (linha != 1) printf("*"); /* Evita dupla impressão
caso lado=1 */
            }

            /* Processamento das linhas internas */

```

```

        if (linha != 1 && linha != lado) {
            if (coluna == 1 || coluna == lado) {
                printf("*");
            } else {
                printf(" ");
            }
        }
        coluna++;
    }
    printf("\n");
    linha++;
}

return 0;
}

```

Boas Práticas de Programação 3.2: O aninhamento de estruturas lógicas simples baseadas em coordenadas cartesianas virtuais (linha, coluna) substitui com extrema elegância a necessidade de matrizes complexas de armazenamento na memória para problemas puramente visuais.

As resoluções prosseguirão na Parte 3, cobrindo o bloco final de exercícios compreendidos entre 3.36 e 3.48.